

# CineSeat — Backend Cheat Sheet

Code-First · Entity Framework Core 10 · PostgreSQL · Onion Architecture — Staj sunumu hazırlık notu

## 1 · Mimari Tipleri (Özet)

| Mimari                         | Bağımlılık yönü                                                         | Ne zaman                                                       |
|--------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------|
| <b>N-Layered</b> (Katmanlı)    | Aşağı: API → İş → Veri → DB. İş mantığı veritabanına bağımlı.           | Basit CRUD, küçük proje, hız önceliği.                         |
| <b>Onion</b> (bizim seçimimiz) | İçeri: API → Infrastructure → Domain. Domain hiçbir şeye bağımlı değil. | Gerçek iş kuralları olan, test edilebilirlik istenen projeler. |
| Clean / Hexagonal              | Onion ile aynı fikir (merkez izole), farklı isimlendirme.               | Onion'ın kardeşleri — aynı prensip.                            |

**Tek fark:** Bağımlılık yönü. N-Layered'da iş mantığı veritabanına bağımlıdır; Onion'da tam tersi — **veritabanı, iş mantığına bağımlıdır**. Onion'ın kuralı: bağımlılıklar hep **içeri (Domain'e) doğru** akar, Domain dışarıyı bilmez.

### Neden Onion'ı seçtik?

Domainimizde korunması gereken gerçek iş kuralları var: **koltuk kilitleme** (aynı koltuğu 2 kişi alamaz), **fiyatlandırma** (öğrenci/çocuk indirimi), **kampanya, rol-yetki (RBAC)**. Bu kurallar veritabanından bağımsız ve test edilebilir olsun istedik. Basit bir CRUD olsaydı N-Layered yeterdi; bizimki değil.

## 2 · Onion Architecture — Bizde Ne Var

| Proje (katman)                  | İçinde ne var                                                                   | Durum |
|---------------------------------|---------------------------------------------------------------------------------|-------|
| <b>CineSeat.Domain</b> (merkez) | Entity sınıfları (21 tablo), Enum'lar, BaseEntity. Hiçbir dışa bağımlılığı yok. | HAZIR |
| <b>CineSeat.Application</b>     | İş mantığı / servisler / use-case'ler. (İleride dolacak.)                       | BOŞ   |
| <b>CineSeat.Infrastructure</b>  | DbContext, Migrations — veritabanıyla konuşan katman.                           | HAZIR |
| <b>CineSeat.Api</b>             | Giriş noktası; DI/bağlantı ayarları (Program.cs), Controller'lar.               | HAZIR |

```
// Bağımlılık yönü (ok = "referans eder / bilir")
API  →  Infrastructure  →  Domain  ←  (herkes Domain'i bilir)
                        ▲
                        Domain hiç kimseyi bilmez ← Onion'ın kalbi
```

**Dürüst durum:** Şu an Onion'ın **iskeletini** kurduk — katmanlar ve bağımlılık yönü doğru, Domain korunuyor, veri katmanı tamam. Application katmanı (servisler) henüz boş; Onion'ın asıl faydası (izole test edilebilir iş kuralları) iş mantığını yazmaya başlayınca ortaya çıkacak.

## 3 · CQRS (Command Query Responsibility Segregation)

**Fikir:** Yazma işlemleri (**Command** — "bir şeyi değiştir") ile okuma işlemleri (**Query** — "bir şey göster") **ayrı yollardan** geçer.

| Command (yazma)                                 | Query (okuma)                               |
|-------------------------------------------------|---------------------------------------------|
| Rezervasyon oluştur, koltuk kilitle, yorum ekle | Film listele, seans göster, yorumları getir |
| Ağır iş kuralı (doğrulama, kilit, fiyat)        | Kural yok, hız önemli — doğrudan veri       |
| CreateReservationCommand → Handler              | GetMoviesQuery → Handler                    |

**Onion'da nasıl oturur:** Command/Query handler'ları **Application** katmanında yaşar. Query'ler ağır Domain doğrulamasından geçmeden hızlı okuma yapar; Command'lar tam kural setinden geçer.

**Bizde hafif (lightweight) CQRS:** Sadece "command ve query kodlarını ayrı sınıflara ayırma" kadarını alınız. **Ayrı okuma veritabanı / event sourcing YOK** — tek PostgreSQL'imiz var, o karmaşıklık bizim ölçeğimizde gereksiz.

**HENÜZ YOK** Bu bir sonraki aşama planı; şu an sadece veri katmanı kuruldu.

## 4 · Code-First Approach

Önce C# sınıflarını yazarız, veritabanı tablolarını o sınıflardan EF Core **otomatik üretir**. (Tersi = Database-First: önce DB kurulur, kod ondan türetilir.)

C# entity yaz → DbContext'e ekle → migration üret → veritabanına uygula  
(Domain) (Infrastructure) (dotnet ef ...) (dotnet ef ...)

**Özü:** Tabloyu SQL ile elle yazmadık; C# sınıfını yazdık, SQL'i EF üretti.

## 5 · Migration Nedir

**Versiyonlanmış şema değişikliği.** Her migration = "veritabanını bir sonraki hale getir" talimatı. Git'e girer, ekipteki herkes aynı DB yapısını alır, elle SQL yazılmaz. Bizde 3 tane: `InitialCreate`, `CompleteDatabaseDesign`, `FixSchemaIssues`.

| Komut                                              | Ne yapar                                                                          |
|----------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>dotnet ef migrations add &lt;isim&gt;</code> | Entity değişikliğini algılar, migration <b>dosyası üretir</b> (DB'ye DOKUNMAZ).   |
| <code>dotnet ef database update</code>             | Migration'daki komutları <b>gerçek PostgreSQL'e uygular</b> (DB yoksa oluşturur). |

### Migration dosyasının anatomisi (önemli — kod detayı)

```
public partial class FixSchemaIssues : Migration
{
    // Up(): migration'ı İLERİ uygular (database update dediğinde çalışır)
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(name: "tech_id", table: "hall_techs");
        migrationBuilder.CreateIndex(name: "ix_users_email",
            table: "users", column: "email", unique: true);
    }

    // Down(): migration'ı GERİ alır (rollback — bir önceki hale döndürür)
    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropIndex(name: "ix_users_email", table: "users");
        migrationBuilder.AddColumn<long>(name: "tech_id", table: "hall_techs", ...);
    }
}
```

| Öge                           | Ne işe yarar                                                                                                                                                                                                                                                                                                                                               |
|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Up()</code>             | <b>İleri yön.</b> <code>database update</code> çalışınca bu metod çalışır — tabloyu/kolonu/index'i <b>oluşturur/değiştirir</b> .                                                                                                                                                                                                                           |
| <code>Down()</code>           | <b>Geri yön (rollback).</b> Migration'ı geri almak istersen ( <code>database update &lt;önceki&gt;</code> ) bu çalışır — Up'ın yaptığını <b>tersine çevirir</b> . Örn. Up bir kolon eklediyse, Down onu siler.                                                                                                                                             |
| <code>MigrationBuilder</code> | Migration içindeki <b>yardımcı nesne</b> . Veritabanı işlemlerini C# metodlarıyla ifade etmeyi sağlar (SQL yazmadan). Metodları: <code>CreateTable</code> , <code>DropColumn</code> , <code>CreateIndex</code> , <code>AddColumn</code> , <code>AddForeignKey</code> , <code>Sql()</code> (ham SQL). EF bunları çalışma anında PostgreSQL SQL'ine çevirir. |
| <code>partial class</code>    | Sınıfın birden fazla dosyaya bölünebildiğini belirtir (EF, ayrı bir <code>.Designer.cs</code> dosyası da üretir; ikisi aynı sınıfın parçası).                                                                                                                                                                                                              |

**ModelSnapshot nedir?** `ApplicationDbContextModelSnapshot.cs` = şemanın o anki **tam fotoğrafı**. EF, yeni migration üretirken "entity'ler ile snapshot arasındaki fark ne?" diye bakar. Bu yüzden snapshot'a elle dokunulmaz, git'e girer.

## 6 · Data Seeding Nedir

Veritabanına **başlangıç/referans verisini** otomatik doldurma. Örn. roller ( `admin`, `member`, `guest` ), izinler, film türleri — uygulama çalışmadan önce hazır olması gereken sabit veriler. Böylece boş DB'ye elle veri girmezsin.

## EF Core'da nasıl yapılır (HasData)

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Role>().HasData(
        new Role { Id = 1, Name = "admin" },
        new Role { Id = 2, Name = "member" },
        new Role { Id = 3, Name = "guest" }
    );
}
```

HasData ile eklenen veri, bir **migration** içine gömülür; `database update` 'te tablolara **INSERT** edilir. Yani seed verisi de versiyonlanır.

**BİZDE HENÜZ YOK** Data seeding'i şu an eklemedik — bir sonraki adımda rolleri, izinleri, film türlerini seed edebiliriz. Sunumda: "Kavramı biliyoruz, HasData ile ekleyeceğiz" diyebilirsiniz.

## 7 · Bilmemiz Gereken Kod Detayları

### Entity → Tablo eşlemesi

| C#                                              | Veritabanı                                     |
|-------------------------------------------------|------------------------------------------------|
| Sınıf ( User )                                  | Tablo ( users )                                |
| Property ( Email )                              | Sütun ( email )                                |
| string / long / bool / decimal / DateTimeOffset | text / bigint / boolean / numeric / timestampz |

### PascalCase → snake\_case

C# geleneği **PascalCase** ( `CreatedAt` ), PostgreSQL geleneği **snake\_case** ( `created_at` ). Program.cs'teki `.UseSnakeCaseNamingConvention()` bu çeviriyi otomatik yapar. Olmasaydı sütunlar `"CreatedAt"` gibi tırnaklı, case-duyarlı ve çirkin olurdu.

### DbContext + DbSet

```
public class ApplicationDbContext : DbContext
{
    // Her DbSet = bir tablo
    public DbSet<User> Users { get; set; } // → "users" tablosu
    public DbSet<Movie> Movies { get; set; } // → "movies" tablosu
}
```

### OnModelCreating + Fluent API

Convention'ın (otomatik kural) bilemediği şeyleri elle tanımladığımız yer:

```
modelBuilder.Entity<User>().HasIndex(u => u.Email).IsUnique(); // benzersiz e-posta
modelBuilder.Entity<SeatLock>().HasIndex(s => new { s.ShowtimeId, s.SeatId }).IsUnique(); // bileşik unique
modelBuilder.Entity<Ticket>().Property(t => t.Price).HasPrecision(10, 2); // numeric(10,2)
```

|                                  |                                                                                     |
|----------------------------------|-------------------------------------------------------------------------------------|
| <code>HasIndex(...)</code>       | Sütun(lar)a index ekler (arama hızı + benzersizlik için).                           |
| <code>.IsUnique()</code>         | O index'i <b>benzersiz</b> yapar → tekrar eden değer engellenir.                    |
| <code>.HasPrecision(10,2)</code> | Ondalık tipi <code>numeric(10,2)</code> yapar (para için — yuvarlama hatası olmaz). |

### Navigation property vs FK property

```
public long TechnologyId { get; set; } // FK property → "technology_id" SÜTUNU
public Technology Technology { get; set; } // Navigation → SÜTUN DEĞİL, ilişkili nesneye erişim
```

**Düzelttiğimiz bug buydu:** İsim `TechId` iken EF onu `Technology` 'nin FK'si sanmadı (konvansiyon `TechnologyId` bekler), ayrıca kendi `technology_id` 'sini yarattı → 2 kolon. `TechnologyId` yapınca EF tanıdı, tek kolon kaldı.

### BaseEntity — kalıtım (inheritance)

```
public class User : BaseEntity // ": BaseEntity" = ortak alanları miras al
```

Id, CreatedAt, UpdatedAt, IsDeleted her tabloda tekrar yazılmaz — BaseEntity 'den gelir. ( IsDeleted = soft delete: satır gerçekten silinmez, "silindi" işaretlenir; geçmiş kayıtlar korunur.)

## Program.cs — bağlantı kurulumu

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection"))
    .UseSnakeCaseNamingConvention());
```

|                                          |                                                            |
|------------------------------------------|------------------------------------------------------------|
| AddDbContext                             | DbContext'i uygulamaya tanıtır (Dependency Injection).     |
| UseNpgsql(...)                           | "PostgreSQL kullan, bağlantı dizesini appsettings'ten al." |
| GetConnectionString("DefaultConnection") | appsettings.json'daki bağlantı bilgisini okur.             |

## 8 · NuGet Paketleri & Terminal Komutları

### Paketler (nerede / ne)

| Paket                                 | Görev                                   |
|---------------------------------------|-----------------------------------------|
| Npgsql.EntityFrameworkCore.PostgreSQL | PostgreSQL sağlayıcısı (Infrastructure) |
| EFCore.NamingConventions              | snake_case (Infrastructure)             |
| Microsoft.EntityFrameworkCore.Design  | Migration üretimi (Api)                 |
| Microsoft.EntityFrameworkCore.Tools   | Migration araçları (Api)                |
| dotnet-ef                             | CLI aracı (dotnet-tools.json)           |

### Komutlar

```
# migration üret
dotnet ef migrations add <isim> ^
--project CineSeat.Infrastructure ^
--startup-project CineSeat.Api

# veritabanına uygula
dotnet ef database update ^
--project CineSeat.Infrastructure ^
--startup-project CineSeat.Api
```

--project : migration'ın yazılacağı proje (DbContext orada). --startup-project : ayarların/bağlantının olduğu çalıştırıcı proje.

**EF Core çalışma mantığı:** Kod tarafında LINQ yazarsın ( context.Movies.Where(...) ) → EF Core bunu SQL'e çevirir → Npgsql PostgreSQL'e gönderir → dönen satırları tekrar C# nesnesine dönüştürür. Sen SQL yazmazsın.

## 9 · Muhtemel Sorular → Hazır Cevaplar

| Soru                    | Cevap                                                                                            |
|-------------------------|--------------------------------------------------------------------------------------------------|
| Code-First nedir?       | Önce C# sınıfı, tablolar ondan otomatik üretilir. Tersi Database-First.                          |
| Migration ne işe yarar? | Şema değişikliğini versiyonlar; ekip aynı DB'yi alır, elle SQL yazılmaz.                         |
| add vs update farkı?    | add dosya üretir (DB'ye dokunmaz), update DB'ye uygular.                                         |
| Up() / Down() ?         | Up ileri uygular, Down geri alır (rollback).                                                     |
| Hangi ORM / mimari?     | EF Core 10 + Npgsql; Onion mimarisi.                                                             |
| DbContext nerede?       | CineSeat.Infrastructure/Data/ApplicationDbContext.cs                                             |
| Neden Onion?            | Karmaşık iş kuralları (kilit/fiyat/yetki) veritabanından bağımsız ve test edilebilir olsun diye. |